

Report primo progetto di Metodi del calcolo scientifico

Daniele Besozzi 894998
Andrea Consonni 900116
Matteo Cervini 902225

18 maggio 2026

Indice

1	Presentazione degli ambienti	2
1.1	Matlab	2
1.2	C++	2
1.2.1	Eigen	2
1.2.2	CHOLMOD	4
1.2.3	Formato Matrix Market	4
1.2.4	Cross platform Make (CMake)	5
1.3	Windows	6
1.4	Linux	6
2	Tempo di esecuzione	7
3	Utilizzo della memoria	8
4	Precisione delle soluzioni	9
5	Conclusioni	10

Capitolo 1

Presentazione degli ambienti

Le misure effettuate per questo progetto sono state eseguite su una macchina con le seguenti caratteristiche:

- Processore: AMD ryzen 7 5700G (8 core, 16 thread, 3.8 GHz)
- RAM: 32 GB DDR4 3200 MHz
- Sistemi operativi: Windows 11 25H2 e Linux Fedora 44
- Linguaggi di programmazione: C++ e Matlab
- Dischi: SSD NVMe

1.1 Matlab

Per quanto riguarda Matlab, abbiamo utilizzato la versione R2025b. Per la risoluzione dei sistemi di equazioni è stato utilizzato il comando `mldivide`, che implementa una serie di controlli per scegliere il metodo più adatto a seconda del tipo di matrice (densa, sparsa, simmetrica, ecc.). Questa libreria viene mantenuta e aggiornata regolarmente da MathWorks, ottimizzando i metodi di risoluzione per alcuni tipi di matrici e migliorando le prestazioni complessive. La documentazione ufficiale di Matlab fornisce i dettagli sulla selezione del metodo di risoluzione in base alle caratteristiche della matrice, dividendo in due macro categorie: matrici dense e matrici sparse. In particolare, in figura 1.1 possiamo vedere il processo di selezione del metodo di risoluzione per matrici sparse, che è quello che abbiamo utilizzato per i nostri benchmark.

1.2 C++

Lo sviluppo del programma in C++ ha richiesto l'utilizzo di diverse librerie, con l'obiettivo di garantire un confronto equo con il programma sviluppato per Matlab. Questa sezione andrà ad illustrare le principali tecnologie e librerie adottate, descrivendone brevemente le caratteristiche, lo stato di manutenzione e la qualità della relativa documentazione.

1.2.1 Eigen

Eigen è una libreria open source per l'algebra lineare che offre una vasta gamma di classi e funzioni per la manipolazione di matrici e vettori, la rappresentazioni di diversi tipi di oggetti matematici e per la risoluzione di sistemi di equazioni lineari. La libreria è stata progettata per essere semplice da installare, efficiente e affidabile. **Eigen** permette inoltre di interfacciarsi con diversi moduli esterni per la risoluzione di sistemi di equazioni lineari, come, ad esempio, il pacchetto **SuiteSparse**, il quale è stato utilizzato per i benchmark presentati in questo report. In particolare, abbiamo utilizzato la libreria **CHOLMOD** di **SuiteSparse**, la quale offre un metodo di risoluzione tramite fattorizzazione di Cholesky più efficiente rispetto al risolutore standard con metodo di Cholesky offerto da **Eigen** [2]. L'interfaccia di **Eigen** per **CHOLMOD**, contenuta nel modulo **CholmodSupport** [3], permette di sfruttare le diverse funzionalità offerte dalla libreria. **Eigen** è attivamente supportata e mantenuta, con l'ultima versione stabile della libreria

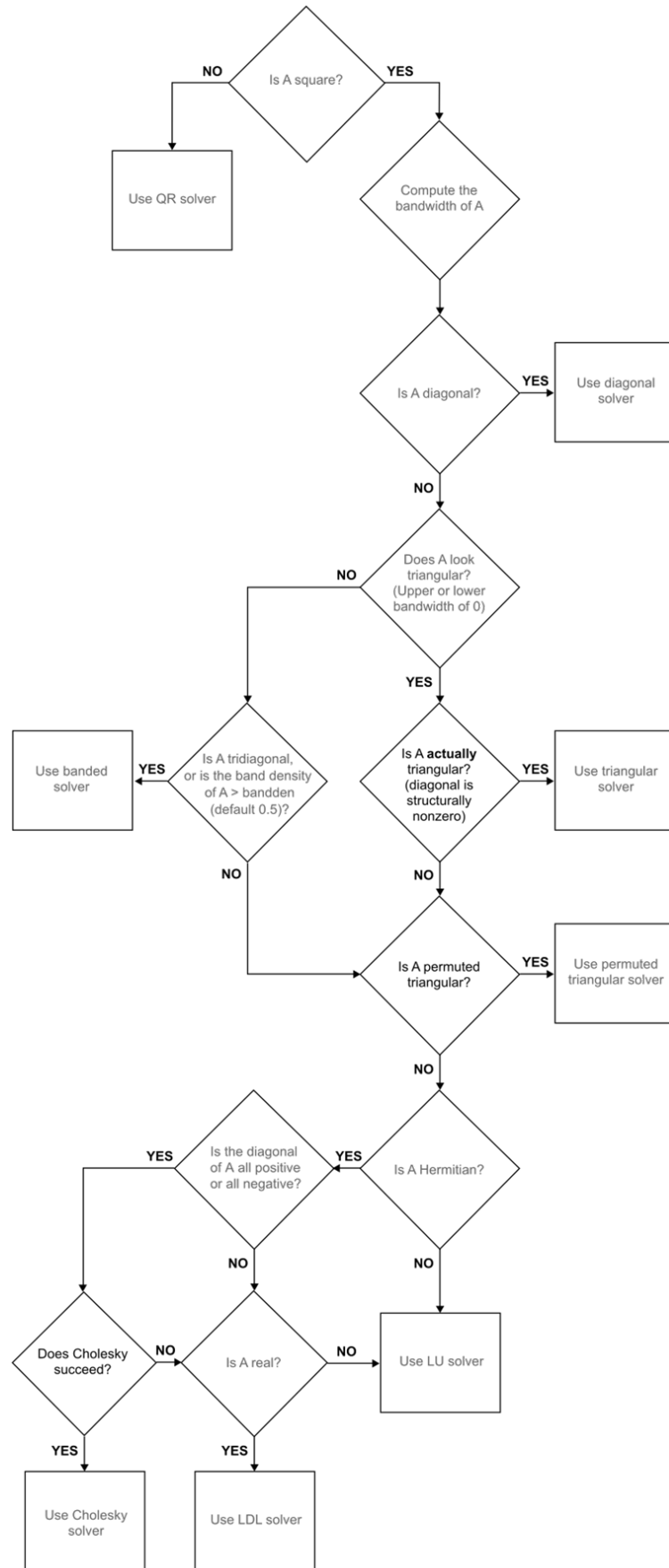


Figura 1.1: Scelta del metodo di risoluzione in Matlab per matrici sparse di `mldivide` [1].

(5.0.0) rilasciata il 30 settembre 2025. La documentazione di Eigen è di ottima qualità e include sia tutorial introduttivi su installazione e utilizzo, sia sezioni più approfondite sul funzionamento interno della libreria e sulla sua integrazione con altre tecnologie e solver.

1.2.2 CHOLMOD

CHOLMOD è una libreria, facente parte del pacchetto **SuiteSparse**, che permette di svolgere efficientemente diverse operazioni su matrici sparse. In particolare, la libreria offre funzioni per la fattorizzazione di matrici sparse, simmetriche e definite positive tramite il metodo di Cholesky e per la risoluzione di sistemi lineari. La libreria è scritta nel linguaggio di programmazione C. Sia A una matrice sparsa, simmetrica e definita positiva; allora la sua decomposizione ottenuta usando il metodo di Cholesky è $A = LL^T$, con L una matrice triangolare inferiore. CHOLMOD implementa due tipi di risolutori di sistemi di equazioni lineari sparsi che sfruttano questa decomposizione: uno **supernodale** e uno **non-supernodale** [4]. In particolare, abbiamo scelto di utilizzare l'implementazione **supernodale** dell'algoritmo di risoluzione, il quale è accessibile in Eigen costruendo un risolutore della classe `CholmodSupernodalLLT` [5]. Gli algoritmi supernodali si basano sulla decomposizione della matrice A in termini di blocchi di colonne, chiamati **supernodi**. L'algoritmo implementato da CHOLMOD è composto da tre fasi di calcolo principali:

1. La prima analizza la struttura della matrice e riordina le righe e le colonne per minimizzare il fill-in, ovvero l'introduzione di elementi non-nulli in L che non compaiono nella posizione corrispondente in A . Il riordinamento della matrice è un'importante fattore di performance, dato che con meno valori non nulli il numero di operazioni necessario e il peso in memoria diminuisce.
2. La seconda fase fattorizza la matrice applicando operazioni sui supernodi
3. La terza risolve due sistemi triangolari di equazioni: $Ly = b$ e $L^T x = y$ nel caso della fattorizzazione di Cholesky.

Tra le tre fasi, quella di fattorizzazione rappresenta la parte computazionalmente più onerosa del risolutore implementato da CHOLMOD [6]. Per sfruttare al meglio le moderne architetture delle CPU multicore, CHOLMOD utilizza la libreria **Open MultiProcessing (OpenMP)** [7] per implementare la parallelizzazione di alcune delle operazioni necessarie alla risoluzione del sistema di equazioni lineari, permettendo quindi di sfruttare al meglio la CPU e, agli scopi del benchmark, ottenere un confronto più equo con Matlab. Infine, CHOLMOD dipende inoltre da diverse librerie per effettuare i calcoli necessari alla decomposizione e alla risoluzione del sistema di equazioni lineari:

- **Librerie per l'ordinamento della matrice:** Per effettuare l'ordinamento della matrice nella prima fase dell'algoritmo di risoluzione, CHOLMOD può utilizzare diversi algoritmi, come *Approximate Minimum Degree ordering* e METIS [6]. Per questo motivo, CHOLMOD dipende dalle librerie di SuiteSparse AMD, CAMD, COLAMD, CCOLAMD e METIS.
- **Basic Linear Algebra Subroutines (BLAS):** BLAS [8] è un insieme di specifiche open per l'implementazione di funzioni a basso livello per svolgere le comuni operazioni richieste per l'algebra lineare. Esistono diverse implementazioni delle funzioni descritte da BLAS, tra cui **OpenBLAS** e **FlexiBLAS**, entrambe utilizzate per i benchmark presentati in questo report.
- **Linear Algebra PACKage (LAPACK):** LAPACK [9] è una libreria che offre diverse funzioni per la risoluzione di sistemi lineari. La libreria dipende dalla presenza di un'implementazione di BLAS ed è scritta in modo tale che la computazione sia svolta effettuando il più possibile chiamate alle funzioni offerte dall'implementazione di BLAS sottostante.

SuiteSparse e CHOLMOD sono librerie attivamente mantenute. In particolare, CHOLMOD dispone di una documentazione tecnica ampia e dettagliata [7], la quale copre in modo esaustivo sia gli aspetti implementativi sia le modalità d'uso della libreria. Inoltre, CHOLMOD viene descritta dai suoi autori in [4].

1.2.3 Formato Matrix Market

Per mantenere il programma scritto in C++ completamente indipendente da tecnologie o formati proprietari, si è deciso di utilizzare la rappresentazione delle matrici di test nel formato **Matrix Market Exchange Format** [10], un formato di file per la rappresentazione di matrici creato dal **National Institute for Standards and Technology** (NIST) e scritto completamente in caratteri ASCII. Questo formato prevede due tipi diversi di rappresentazione per le matrici:

- **Formato a Coordinate:** Questo formato è adatto per rappresentare generiche matrici sparse. Vengono fornite solamente le entrate non nulle e le loro coordinate all'interno della matrice.
- **Formato ad array:** Questo formato è adatto per rappresentare generiche matrici dense. Vengono fornite tutte le entrate, elencate per colonne.

Un file in formato Matrix Market è composto da 4 parti [11]:

1. **Header Line:** contiene un identificatore e quattro campi testuali. L'header deve essere la prima riga del file e può avere la forma `%MatrixMarket object format field symmetry` oppure la forma `%%MatrixMarket object format field symmetry`. I quattro campi testuali che seguono la prima stringa possono avere i seguenti valori:
 - **object:** il tipo di oggetto rappresentato; può avere valore `matrix`, per le matrici, oppure `vector`, per i vettori
 - **format:** il tipo di rappresentazione contenuta nel file, può avere valore `coordinate` oppure `array`
 - **field:** il tipo di entrate della matrice, può assumere i valori `integer`, `real`, `double`, `complex` o `pattern`. Nel caso di quest'ultimo valore, nel file verranno indicate solamente le coordinate delle entrate non nulle, senza il loro valore.
 - **symmetry:** il tipo di simmetria della matrice, può avere i valori `general` (utilizzabile per i valori `real`, `complex`, `integer` o `pattern` di `field`), `symmetric` (`real`, `complex`, `integer` o `pattern`), `skew-symmetric` (`real`, `complex`, `integer`) o `hermitian` (`complex`)
2. **Comment lines:** permette al creatore del file di inserire commenti
3. **Size line:** indica il numero di righe e colonne della matrice, nonché il numero di entrate non nulle
4. **Data lines:** specifica la posizione delle entrate della matrice e il loro valore

Per permettere un caricamento rapido delle matrici in questo formato, si è deciso di utilizzare la libreria open source `fast_matrix_market` [12], la quale mette a disposizione un modulo veloce e parallelizzato per caricare le matrici sparse direttamente nei tipi di oggetti definiti da `Eigen`. `fast_matrix_market` non è più attivamente sviluppato (l'ultima commit al progetto risale a 2 anni prima della stesura di questo report). Inoltre, la libreria non dispone di una documentazione ufficiale, preferendo invece l'utilizzo di brevi file README che descrivono le funzionalità disponibili, le scelte implementative adottate e alcuni esempi d'uso per sfruttarne al meglio le funzionalità.

1.2.4 Cross platform Make (CMake)

Cross platform Make [13], abbreviato in CMake, è uno strumento di automazione che permette di gestire in maniera semplice il processo di compilazione e build di un progetto. CMake è largamente usato per i linguaggi C e C++, tuttavia esso può essere usato per la gestione di progetti scritti anche in altri linguaggi di programmazione. Il tipico uso di CMake ruota intorno ad uno più file `CMakeList.txt`, comunemente chiamati "list files". In un progetto software gestito con CMake, avremo un list file per ogni cartella per la quale vogliamo specificare le modalità di gestione dei file e quali sono le operazioni necessarie da svolgere al suo interno. Nel progetto C++ sviluppato per questo benchmark, abbiamo utilizzato un singolo list file per la gestione di tutto il progetto, nel quale vengono specificate le seguenti operazioni da svolgere ogni volta che il progetto viene compilato:

1. Se non sono presenti, scaricare le matrici di test in formato `.mtx`
2. Importare tutte le dipendenze necessarie al funzionamento del progetto
3. Specificare come le dipendenze debbano essere linkate al progetto e quali solo le flag di ottimizzazione che il compilatore deve utilizzare
4. Disabilitare le asserzioni interne ad `Eigen` (linee di codice utilizzate per effettuare dei test interni durante l'esecuzione), per migliorare la velocità di esecuzione del programma, ed indicare alla libreria di utilizzare l'implementazione di `BLAS` presente nel sistema
5. Rendere disponibili al programma i percorsi delle cartelle rilevanti per l'esecuzione del progetto, per evitare di indicarli esplicitamente per ogni sistema operativo

CMake è un software costantemente aggiornato e supportato da una documentazione completa e ben strutturata. La documentazione mette a disposizione sia guide introduttive per familiarizzare con lo strumento, sia sezioni di riferimento dettagliate per tutti i comandi offerti.

1.3 Windows

1.4 Linux

Capitolo 2

Tempo di esecuzione

Capitolo 3

Utilizzo della memoria

Capitolo 4

Precisione delle soluzioni

Capitolo 5

Conclusioni

Bibliografia

- [1] *mldivide - Solve systems of linear equations $Ax = B$ for x - MATLAB*. visitato il giorno 14 mag. 2026. indirizzo: <https://it.mathworks.com/help/matlab/ref/double.mldivide.html>
- [2] *Eigen::SimplicialLLT Class Template Reference*. visitato il giorno 16 mag. 2026. indirizzo: https://libeigen.gitlab.io/eigen/docs-nightly/classEigen_1_1SimplicialLLT.html
- [3] *Eigen: CholmodSupport module*. visitato il giorno 15 mag. 2026. indirizzo: https://libeigen.gitlab.io/eigen/docs-5.0/group__CholmodSupport__Module.html
- [4] Y. Chen, T. A. Davis, W. W. Hager e S. Rajamanickam, «Algorithm 887: CHOLMOD, Supernodal Sparse Cholesky Factorization and Update/Downdate,» *ACM Transactions on Mathematical Software*, vol. 35, n. 3, pp. 1–14, ott. 2008, ISSN: 0098-3500, 1557-7295. DOI: 10.1145/1391989.1391995 visitato il giorno 15 mag. 2026. indirizzo: <https://dl.acm.org/doi/10.1145/1391989.1391995>
- [5] *Eigen::CholmodSupernodalLLT Class Template Reference*. visitato il giorno 15 mag. 2026. indirizzo: https://libeigen.gitlab.io/eigen/docs-5.0/classEigen_1_1CholmodSupernodalLLT.html
- [6] V. Le Fèvre, T. Usui e M. Casas, «A Selective Nesting Approach for the Sparse Multi-threaded Cholesky Factorization,» in *2022 IEEE/ACM 7th International Workshop on Extreme Scale Programming Models and Middleware (ESPM2)*, nov. 2022, pp. 1–9. DOI: 10.1109/ESPM256814.2022.00006 visitato il giorno 15 mag. 2026. indirizzo: <https://ieeexplore.ieee.org/document/10029458>
- [7] T. Davis, *CHOLMOD User guide*, lug. 2025. visitato il giorno 17 mag. 2026. indirizzo: https://github.com/DrTimothyAldenDavis/SuiteSparse/blob/dev/CHOLMOD/Doc/CHOLMOD_UserGuide.pdf
- [8] *BLAS (Basic Linear Algebra Subprograms)*. visitato il giorno 16 mag. 2026. indirizzo: <https://www.netlib.org/blas/>
- [9] *LAPACK-Linear Algebra PACKage*. visitato il giorno 16 mag. 2026. indirizzo: <https://www.netlib.org/lapack/>
- [10] *Matrix Market: File Formats*. visitato il giorno 17 mag. 2026. indirizzo: <https://math.nist.gov/MatrixMarket/formats.html>
- [11] *Matrix Market File Format Documentation*. visitato il giorno 17 mag. 2026. indirizzo: <https://networkrepository.com/mtx-matrix-market-format.html>
- [12] A. Lugowski, *fast_matrix_market: Fast and Full-Featured Matrix Market I/O Library*, gen. 2023. DOI: 10.5281/zenodo.10223767 visitato il giorno 17 mag. 2026. indirizzo: https://github.com/alugowski/fast_matrix_market
- [13] Kitware, Inc., *CMake Reference Documentation*, Accessed: 2026-05-17, 2026. indirizzo: <https://cmake.org/cmake/help/latest/>